

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL INTEGRADOR

CLASE 01 • NIVEL 4 - INTEGRADOR

Clase 01: Arquitectura de Software y Planificación del Proyecto

«Diseñar los Planos de un Edificio Antes de Poner el Primer Ladrillo»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)
Rol: AI Solutions Architect & Principal Software Engineer
Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 01** del **Curso 4: Taller Práctico & Proyecto Final Integrador**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender la arquitectura en capas (Clean Architecture / MVC), contratos de API y selección de Track.

Competencia Práctica: Diseñar el diagrama de componentes, definir el modelo de datos y estructurar el repositorio del proyecto final.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 01: Arquitectura de Software y Planificación del Proyecto

Un proyecto de software profesional comienza con una arquitectura sólida que garantiza escalabilidad y mantenimiento.

☀ Metáfora Central: Diseñar los Planos de un Edificio Antes de Poner el Primer Ladrillo

Diseñar el software es como dibujar los planos estructurales de una casa: define dónde irán las tuberías (APIs) y los cimientos (BD).

Principios Teóricos y Modelo Mental

Separación de responsabilidades: Frontend (Presentación), Backend (Lógica de Negocio) y Base de Datos (Persistencia).

Definición de contratos API-First mediante esquemas OpenAPI y modelos Pydantic.

⚡ Regla de Oro en Python

Nunca empieces a codificar sin tener un diagrama de arquitectura y las entidades de datos definidas.

Arquitectura y Movimiento de Datos en Memoria

Arquitectura general del sistema distribuido cliente-servidor.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Diseño de la capa de presentación (Streamlit UI / Web).	Vistas y componentes maquetados.
2. Evaluación	Especificación de endpoints REST en FastAPI.	Contratos DTO / Pydantic listos.
3. Transformación	Modelado de base de datos relacional y transacciones.	Esquema de tablas DDL.
4. Retorno / Salida	Integración del motor de Inteligencia Artificial.	Pipeline de agentes conectado.

Visualización Mental

Cada capa solo debe comunicarse con sus capas adyacentes a través de interfaces bien definidas.

Código de Demostración en Python 3.10+

Estructura modular del proyecto y modelos de dominio:

main.py (Python 3.10+)

PEP 8 Compliant

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class ProyectoConfig(BaseModel):
    nombre_app: str = "Wisrovi Enterprise App"
    version: str = "1.0.0"
    debug: bool = False

class ItemDTO(BaseModel):
    id: Optional[int] = None
    titulo: str
    creado_en: datetime = datetime.now()

config = ProyectoConfig()
print(f"Iniciando arquitectura para: {config.nombre_app} v{config.version}")
```

Análisis de Ingeniería del Código

Uso de configuración fuertemente tipada y Data Transfer Objects (DTO) con Pydantic.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Antipatrón: Código espagueti sin separación de capas.

⚠ Gotcha Frecuente (Trampa de Principiante)

Colocar consultas SQL directamente dentro de los componentes visuales del frontend destruye la mantenibilidad.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

En archivo del frontend:
cursor.execute('INSERT INTO...') ❌
Acoplamiento peligroso

✅ Patrón Correcto

Frontend -> Llama a API REST -> API invoca
Repositorio -> BD ✅

🛡 Consejo de Resiliencia en Producción

Organiza tu código en carpetas 'domain', 'services', 'repositories' y 'api'.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 01: Arquitectura de Software y Planificación del Proyecto**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Diseñar los Planos de un Edificio Antes de Poner el Primer Ladrillo
2. Regla de Oro	Nunca empieces a codificar sin tener un diagrama de arquitectura y las entidades de datos definidas.
3. Gotcha Crítico	Colocar consultas SQL directamente dentro de los componentes visuales del frontend destruye la mantenibilidad.
4. Buenas Prácticas	Organiza tu código en carpetas 'domain', 'services', 'repositories' y 'api'.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Dibuja el diagrama de arquitectura y redacta las 5 rutas principales de tu API.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.